

Trabajo Práctico 6 — Test Driven Development

Práctico 13 · EcoHarmony Park · User Story: Inscribirme a actividad

Ingeniería y Calidad de Software — 4K4 (2026)

Grupo 2 (número par)

Docentes: Ing. Constanza Garnero · Ing. Laura Covaro

Ayudante: Salvador Barbera

Fecha de entrega: junio 2026

Repositorio: github.com/Martinrinaudo/ISW_Grupo2_4k4_2026

Código: Trabajos Prácticos/Trabajos Prácticos Grupales/TP6/ecoharmony-tdd/

Integrantes

Legajo	Apellido y nombre
90639	Conterno, Tomás
95680	de la Vega, Federico Agustín
98292	Eggel, Tomás Agustín
93052	Escobar, Máximo Joaquín
91572	Escudero, Francisco Javier
91227	Formoso, Matias Agustin
95403	Franco, Martina
94192	Padilla, Tomas Lautaro
95738	Paglino, Luciano Ivo
89615	Rinaudo, Martín
97924	Zin, Cecilia Belén

Trabajo grupal — ISW Grupo 2

1. Entregables de la consigna

El Práctico 13 (TP6 TDD) pide:

- ☒ Reglas de estilo de código elegidas en el Trabajo Práctico 2 (Requerimientos Ágiles — User Stories y Estimaciones).
- ☒ Implementación funcional de la user story asignada al grupo par: **Inscribirme a actividad**.
- ☒ Conjunto de pruebas automatizadas que cubran los casos definidos (21 tests con Vitest, todos en verde).
- ☒ Documento de diseño que justifique las decisiones tomadas (este archivo).

Stack: Next.js 15, TypeScript, Vitest. Pantalla: /inscripcion.

2. User story

Frase verbal: Como visitante quiero inscribirme a una actividad para reservar mi lugar en la misma.

Story points (TP2): 5

Criterios de aceptación

#	Criterio
1	Seleccionar actividad (Tirollesa, Safari, Palestra, Jardinería) con cupos en el horario elegido.
2	Seleccionar horario dentro de los disponibles.
3	Indicar cantidad de participantes.
4	Por participante: nombre, DNI, edad y talla si la actividad lo exige.
5	Aceptar términos y condiciones específicos de la actividad.
6	Al finalizar, enviar correo con la información de la inscripción.

3. Reglas de negocio

Valores confirmados por cátedra (mail del ayudante, junio 2026). Están en `lib/ecoharmony/constantes.ts`.

Tema	Regla
Parque	Abierto 8:30 a 19:00. Cerrado lunes, 25/12 y 1/1.
Actividades	Horario 9:00 a 18:00; un turno por hora. No hay turno que empiece a las 18:00 (último inicio: 17:00).
Cupos por turno	Safari 8 · Palestra 12 · Jardinería 12 · Tirolesa 10
Edad mínima	Safari y Jardinería: sin mínimo. Palestra ≥ 12 años. Tirolesa ≥ 8 años.
Anticipación	24 horas antes del inicio del turno (fecha + hora), no dos días calendario.
Talle	Obligatorio en Palestra y Tirolesa (XS a XXL).
Términos y condiciones	Modal y textos distintos por actividad (TYC_POR_ACTIVIDAD).

4. Arquitectura

Separación en tres capas para no duplicar reglas entre pantalla y dominio:

Capa	Ubicación	Responsabilidad
Dominio	lib/ecoharmony/	Validaciones, cupos en memoria, servicio de inscripción, contenido del mail, puerto Mailer.
Aplicación / UI	app/inscripcion/, components/	Formulario, modal TyC, server actions; delega en el servicio.
Tests	tests/	Vitest solo sobre backend (pedido de cátedra). Reloj inyectable.

Archivos principales: `constantes.ts`, `reglas.ts`, `inscripcion-service.ts`, `memoria.ts`, `mailer-contenido.ts`, `app/inscripcion/actions.ts`, `InscripcionForm.tsx`, `TyCModal.tsx`.

Flujo al confirmar inscripción

```
validar solicitud → enviar mail → descontar cupos
```

Si el envío de mail falla, no se descuentan cupos. En tests se usa `MailerRegistro`; en la app, SMTP con `.env.local` o `MailerConsola` si no hay credenciales.

Reloj: `RelojFijo` en tests (fecha 20/05/2026 09:00 en `tests/helpers.ts`), `RelojSistema` en producción/demo.

5. Metodología TDD

Ciclo aplicado sobre el servicio de dominio:

1. **Red:** escribir un test que falla.
2. **Green:** código mínimo para que pase.
3. **Refactor:** mejorar diseño sin romper tests.

Ejemplo — anticipación 24 h: primero el test *regla: menos de 24 hs antes del turno* con turno el mismo día que el reloj fijo (falla con `AnticipacionInvalidaError`); luego *regla: 24 hs antes del turno alcanza* con turno al día siguiente (pasa y envía mail). La constante `ANTICIPACION_MIN_MS` centraliza el valor.

Verificación al entregar: `npm test` — 21 tests passed (3 archivos).

6. Mapa de pruebas automatizadas

6.1 Pruebas de usuario del enunciado (TP2)

Prueba (enunciado)	Resultado esperado	Test automatizado	Archivo
1 — Inscripción completa con mail	Pasa	prueba 1 pasa: inscripción exitosa envía mail	inscripcion.test.ts
2 — Sin cupo en el horario	Falla	prueba 2 falla: sin cupo para el horario	inscripcion.test.ts
3 — Sin talle cuando no hace falta	Pasa	prueba 3 pasa: Safari sin talle	inscripcion.test.ts
4 — Parque cerrado o actividad no disponible	Falla	prueba 4 falla: lunes parque cerrado; prueba 4 falla: no hay turno a las 18:00	inscripcion.test.ts
4 — Último turno válido	Pasa	prueba 4 pasa: último turno 17:00	inscripcion.test.ts
5 — Sin TyC	Falla	prueba 5 falla: sin términos y condiciones	inscripcion.test.ts
6 — Sin talle obligatorio	Falla	prueba 6 falla: Palestra sin talle	inscripcion.test.ts
6 — Con talle obligatorio	Pasa	prueba 6 pasa: Palestra con talle	inscripcion.test.ts

6.2 Tests adicionales y mail

Test	Archivo
regla: edad mínima Tirolesa	inscripcion.test.ts
regla: menos de 24 hs antes del turno	inscripcion.test.ts
regla: 24 hs antes del turno alcanza	inscripcion.test.ts
regla: etiqueta cupos disponibles	inscripcion.test.ts
regla: cantidad mayor a cupos restantes	inscripcion.test.ts
regla: feriado 25 diciembre cerrado	inscripcion.test.ts
asunto incluye la actividad	mailer-contenido.test.ts
incluye actividad, fecha, horario e ID	mailer-contenido.test.ts
envía a el email del visitante con asunto y cuerpo correctos	mail-inscripcion.test.ts
el cuerpo lista todos los participantes con talle si aplica	mail-inscripcion.test.ts
si la inscripción falla, no se envía mail	mail-inscripcion.test.ts
si el mail falla, no consume cupos	mail-inscripcion.test.ts

Total: $15 + 2 + 4 = 21$ tests.

7. Decisiones de diseño

- **Tests en backend:** la lógica vive en `InscripcionService`; la UI no tiene tests E2E en este TP.
- **Mail mockeado:** `MailerRegistro` guarda destinatario, asunto y cuerpo; se valida con `cuerpoCorreo()` compartido con SMTP.
- **SMTP opcional:** quien haga la demo configura `.env.local` (ver docs/CONFIGURACION-MAIL.md); los tests no dependen de red.
- **Cupos en memoria:** al refrescar o reiniciar el servidor de desarrollo los cupos vuelven al máximo; aceptado para la consigna.
- **TyC por actividad:** textos en constantes; modal antes de confirmar.
- **UX cupos:** si la cantidad de personas supera los cupos restantes, el campo se marca en rojo con mensaje (sin test automatizado de UI).
- **Sin validación de DNI duplicado:** la US y la cátedra no lo exigen; una misma persona puede ocupar varios lugares.

8. Cómo ejecutar el proyecto

```
cd ecoharmony-tdd
npm install
npm test
npm run dev
```

Abrir `http://localhost:3000/inscripcion` (o el puerto que indique Next si 3000 está ocupado).

Trabajar en carpeta local; en Google Drive `npm install` suele fallar.

9. Referencias

- Repositorio del grupo: [ISW Grupo2 4k4 2026](#)
- Proyecto TP6: Trabajos Prácticos/Trabajos Prácticos Grupales/TP6/ecoharmony-tdd/
- Resumen en repo: docs/decisiones-tp6-tdd.md
- US: `tp2-ecoharmony/us-inscribir-actividad.md` (material del grupo / apuntes)
- Convención SCM del grupo: entregable PDF `ISW_TP_6.pdf` en carpeta TP6